



SISTEMA DE GESTIÓN DE ÓRDENES DE SERVICIO PARA TALLER MECÁNICO

INFORME TÉCNICO DE ARQUITECTURA Y PATRONES DE DISEÑO

Presentado por: Francisco R. Diaz
Asignatura: Ingeniería de Software
Fecha: 12/06/2026

1. Introducción

El sector automotriz en la República Dominicana cuenta con un gran número de talleres mecánicos independientes que operan con procesos manuales o semi-digitalizados. La gestión de las órdenes de servicio desde la recepción del vehículo hasta la emisión de la factura implica una serie de pasos secuenciales que, si no son controlados adecuadamente, pueden ocasionar errores, pérdida de información, ineficiencia operativa y, en última instancia, la insatisfacción del cliente.

Este proyecto surge como respuesta a esa problemática. El objetivo es desarrollar un sistema web de gestión de órdenes de servicio que digitalice completamente el flujo de trabajo del taller: registro de clientes y vehículos, apertura de órdenes, diagnóstico, elaboración de presupuestos, ejecución de reparaciones y facturación. El sistema contempla además el cálculo automático del ITBIS (Impuesto sobre Transferencia de Bienes Industrializados y Servicios) equivalente al 18%, de acuerdo con la legislación fiscal vigente en el país.

Desde el punto de vista tecnológico, la solución está desarrollada con una arquitectura moderna y robusta: el backend utiliza Node.js con el framework Express, Prisma ORM para la gestión de la base de datos PostgreSQL, y una arquitectura hexagonal (también llamada Puertos y Adaptadores) que garantiza la separación de responsabilidades entre el dominio del negocio y la infraestructura técnica. El frontend está construido con React y Vite. El sistema se despliega en contenedores Docker y está disponible en producción mediante la plataforma Render.

El presente informe tiene como propósito identificar y documentar los patrones de diseño que fueron aplicados durante el desarrollo del sistema, con especial énfasis en el patrón Decorator, que surgió como solución natural a uno de los problemas de diseño más relevantes encontrados durante la implementación. Se incluye además una justificación técnica, el contexto arquitectónico y fragmentos de código real del proyecto.

2. Problema de Diseño Identificado

Durante el análisis funcional del módulo de servicios, se identificó un problema de diseño recurrente en sistemas con jerarquías de clases: la 'explosión combinatoria de subclases'. En el contexto de este taller mecánico, los servicios mecánicos base (Mantenimiento Preventivo, Reparación Correctiva, Diagnóstico Electrónico) pueden requerir características adicionales que varían según la solicitud específica del cliente al momento de la recepción.

Características adicionales frecuentes solicitadas:

- Servicio Express: El cliente requiere entrega rápida con prioridad máxima, lo que implica un cargo adicional.
- Garantía Extendida: Cobertura adicional sobre la mano de obra por un período de tiempo mayor al estándar.
- Servicio a Domicilio: Recogida y entrega del vehículo en la dirección del cliente, con un costo logístico asociado.
- Reporte Fotográfico: Documentación visual del estado del vehículo antes y después de la reparación.

Si se intentara resolver esto usando herencia tradicional, se generaría una cantidad inmanejable de clases. Por ejemplo, combinando solo tres extras con tres tipos de servicio base, se necesitarían potencialmente $3 \times 2^3 = 24$ subclases. Agregar un cuarto extra duplicaría ese número. Este enfoque viola directamente el Principio de Responsabilidad Única (SRP) del SOLID, ya que cada clase combinada tendría múltiples razones para cambiar.

Adicionalmente, la lógica del cálculo del total se duplicaría en cada subclase, haciendo el mantenimiento extremadamente difícil y propenso a errores. La solución óptima, por tanto, no pasa por la herencia, sino por la composición dinámica de comportamientos en tiempo de ejecución.

3. Patrón de Diseño Seleccionado: Decorator

3.1 Nombre y Categoría

Nombre: Decorator (Decorador)

Categoría: Patrón Estructural (Gang of Four — GoF)

Fuente de consulta principal: Refactoring Guru — <https://refactoring.guru/es/design-patterns/decorator>

3.2 Descripción General del Patrón

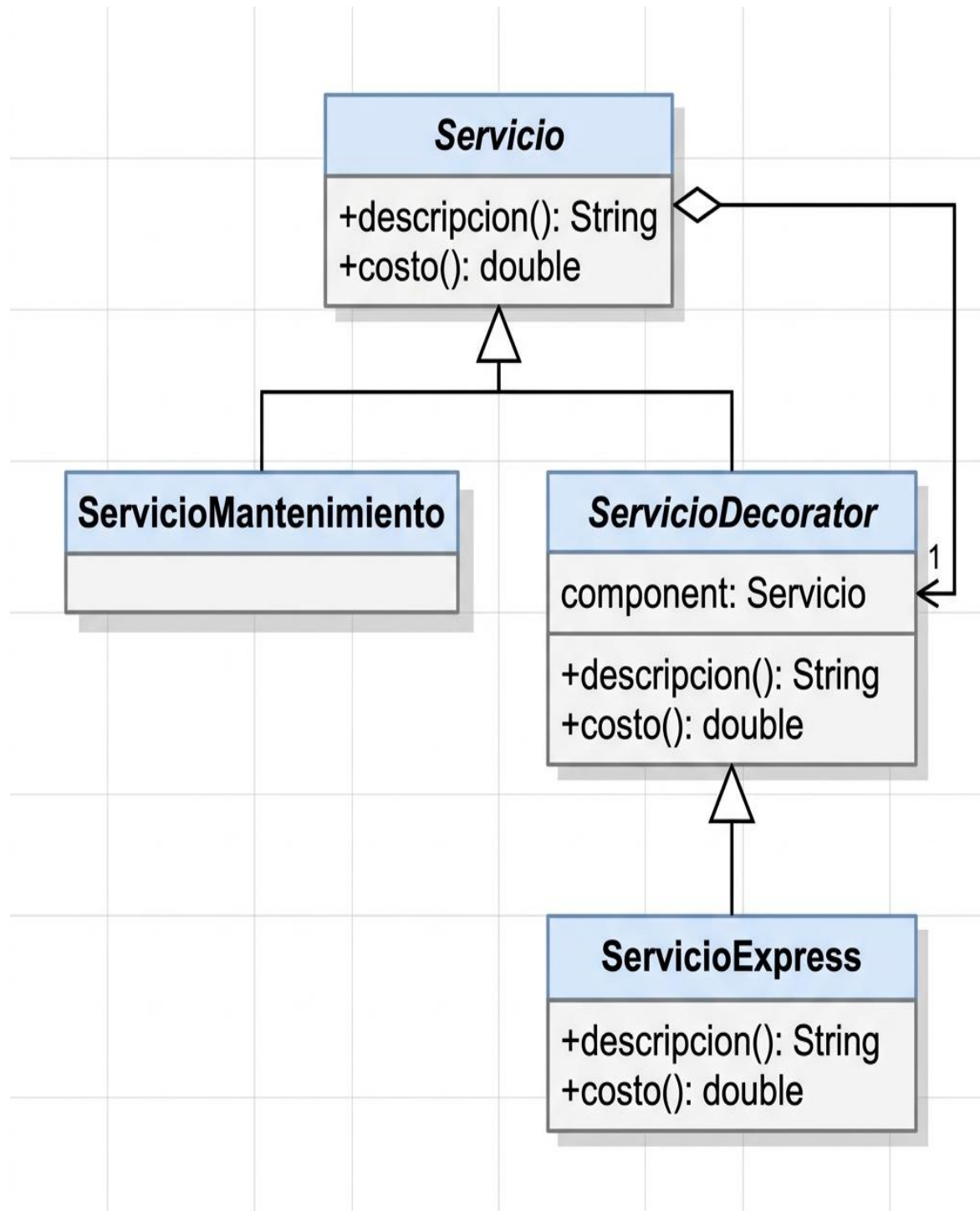
El patrón Decorator es un patrón estructural que permite añadir comportamientos y responsabilidades a un objeto de manera dinámica, envolviéndolo dentro de objetos 'decoradores' que implementan la misma interfaz. A diferencia de la herencia, que extiende el comportamiento a nivel de clase (estáticamente, en tiempo de compilación), el Decorator opera a nivel de objeto y en tiempo de ejecución.

Su estructura se compone de cuatro elementos clave:

1. Component (Componente): Define la interfaz común para el objeto base y los decoradores.
2. ConcreteComponent (Componente Concreto): Es el objeto base que será decorado. Tiene la implementación por defecto.
3. Decorator (Decorador Base): Contiene una referencia al Componente y delega las operaciones a él.
4. ConcreteDecorator (Decorador Concreto): Extiende al Decorator añadiendo nuevas responsabilidades antes o después de delegar.

3.3 Diagrama de Clases

En el diagrama, Servicio actúa como el Component. ServicioMantenimiento, ServicioReparacion y ServicioDiagnostico son los ConcreteComponents. ServicioDecorator es el Decorator Base, y ServicioExpressDecorator es un ConcreteDecorator.



3.4 Implementación en el Proyecto (JavaScript)

// ServicioDecorator.js — Clase base del decorador

```
class ServicioDecorator extends Servicio {  
  constructor(servicio) {  
    super({ descripcion: servicio.descripcion,  
           costo: servicio.costo,  
           tiempoEstimado: servicio.tiempoEstimado });  
    this.componente = servicio;  
  }  
  calcularTotal() { return this.componente.calcularTotal(); }  
  tipo()          { return this.componente.tipo(); }  
}
```

// ServicioExpressDecorator.js — Decorador concreto

```
class ServicioExpressDecorator extends ServicioDecorator {  
  constructor(servicio, cargoExtra = 500) {  
    super(servicio);  
    this.cargoExtra = cargoExtra;  
  }  
  calcularTotal() {  
    return this.componente.calcularTotal() + this.cargoExtra;  
  }  
  get descripcion() {  
    return `${this.componente.descripcion} (EXPRESS)`;  
  }  
}
```

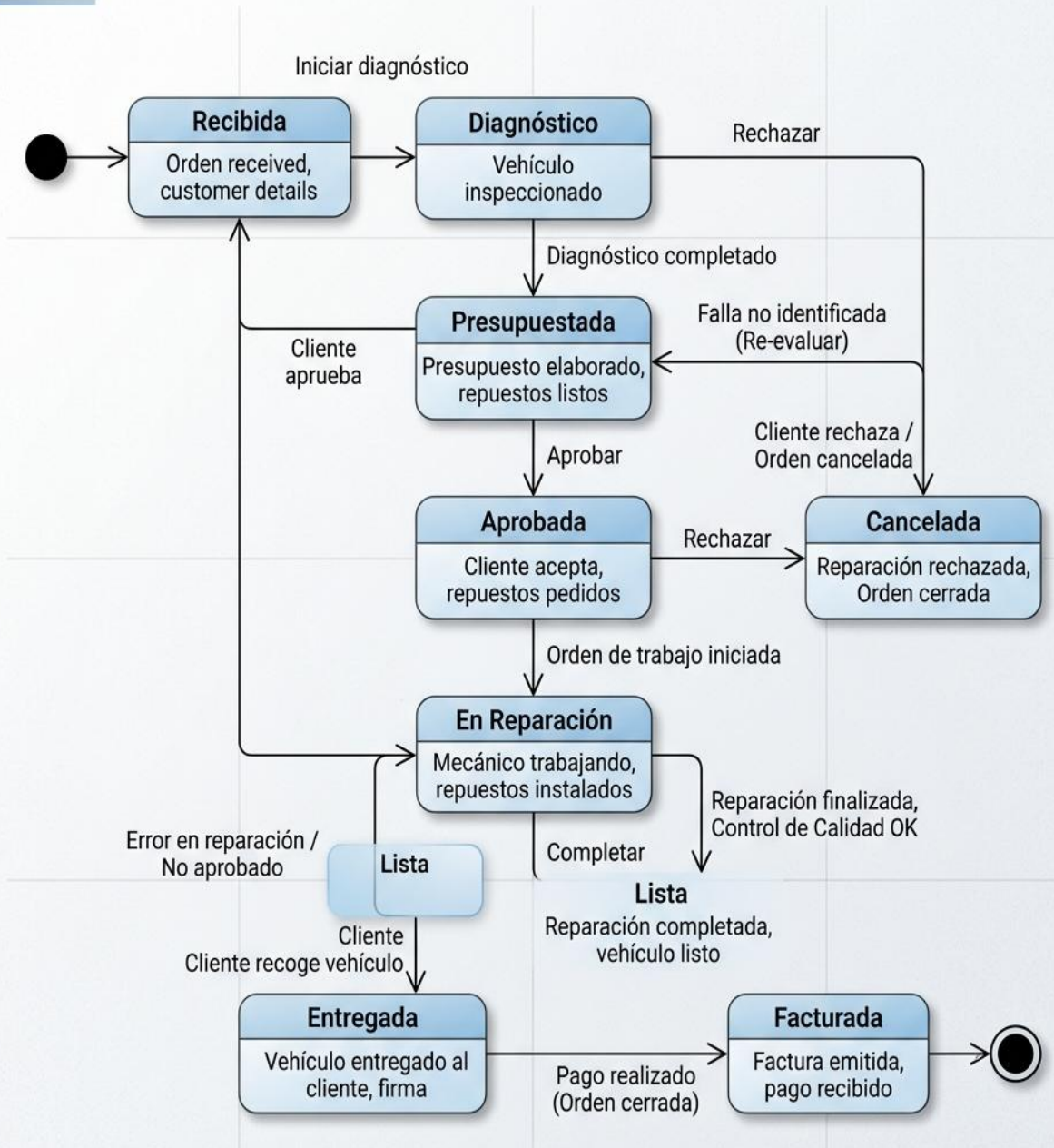
// Uso — Composición dinámica en tiempo de ejecución

```
const cambioAceite = new ServicioMantenimiento('Cambio de aceite 5W-30', 1500, 30);  
const expreso      = new ServicioExpressDecorator(cambioAceite, 500);  
// Total: 2000 RD$ (1500 base + 500 express)
```

3.5 Patrón State (Ciclo de Vida)

La Orden de Servicio es el corazón del sistema. Su ciclo de vida es complejo y está gestionado por el patrón State. Cada estado (Recibida, Diagnosticada, Reparada) es una clase que decide a qué estado puede transicionar.

FLUJO DE ESTADOS DE ORDEN DE SERVICIO DE TALLER MECÁNICO



4. Justificación: ¿Por Qué el Patrón Decorator es el Adecuado?

La elección del patrón Decorator como solución al problema de personalización de servicios responde a un análisis metodológico riguroso basado en los principios SOLID y en las buenas prácticas de la Arquitectura Hexagonal.

a) Principio de Abierto/Cerrado (OCP): Las clases de servicio base (ServicioMantenimiento, ServicioReparacion) están cerradas para modificación, pero abiertas para extensión. Cualquier nuevo 'extra' simplemente requiere crear un nuevo decorador, sin tocar el código que ya está funcionando y probado.

b) Composición sobre Herencia: La herencia establece relaciones rígidas en tiempo de compilación. El Decorator, en cambio, permite combinar comportamientos en tiempo de ejecución según las selecciones del cliente al crear la orden. Esto se alinea perfectamente con la naturaleza dinámica de los pedidos de un taller real.

c) Eliminación de la 'Class Explosion': Como se demostró en la sección del problema, la herencia genera una cantidad exponencial de clases. Con el Decorator, la misma funcionalidad se obtiene con exactamente cuatro clases: la base Servicio, el wrapper ServicioDecorator, y un decorador por cada 'extra' (Express, Garantía, Domicilio). Las combinaciones son ilimitadas con solo esas clases.

d) Principio de Responsabilidad Única (SRP): Cada decorador tiene una única responsabilidad: añadir su cargo extra específico al total. ServicioExpressDecorator solo sabe de prioridad express. ServicioGarantiaDecorator solo sabe de garantías. Esto hace que el código sea fácil de testear de forma unitaria.

e) Compatibilidad con la Arquitectura Hexagonal: Los decoradores viven exclusivamente en la capa de dominio, sin dependencias de infraestructura. Esto preserva la regla fundamental de la arquitectura hexagonal: el dominio no depende de la 'outer layer'.

5. Otros Patrones Implementados en el Proyecto

El Decorator no actúa en aislamiento; forma parte de un ecosistema de patrones que trabajan juntos para garantizar la solidez del sistema:

Patrón	Capa	Propósito en el sistema
State	Dominio	Controla el ciclo de vida de la OrdenServicio (Recibida → Facturada).
Builder	Dominio	Permite construir órdenes complejas paso a paso con múltiples campos opcionales.
Factory Method	Dominio	Crea el tipo correcto de Servicio (Mantenimiento, Reparación) según el parámetro recibido.
Strategy	Dominio	Aplica diferentes algoritmos de descuento (Frecuente 10%, Corporativo, Promoción).
Repository	Infraestructura	Abstrae el acceso a PostgreSQL, permitiendo cambiar de motor sin tocar el dominio.
Observer	Aplicación	Notifica automáticamente (SMS, Email, Bitácora) cuando la orden cambia de estado.
Facade	Aplicación	TallerFacade provee una API simplificada al frontend, ocultando la complejidad interna.

6. Conclusión

El análisis realizado en este informe demuestra que los patrones de diseño no son simplemente una formalidad académica, sino herramientas prácticas de ingeniería de software que resuelven problemas reales de manera elegante y probada. En el caso del Sistema de Gestión de Órdenes de Servicio para Taller Mecánico, el desafío de añadir funcionalidades variables a los servicios mecánicos sin degradar la estructura del código encontró una solución directa y escalable en el patrón Decorator.

La combinación del patrón Decorator con los demás patrones del ecosistema State para el ciclo de vida, Strategy para los descuentos configurables, Observer para las notificaciones desacopladas y Builder para la construcción fluida de órdenes produce una arquitectura de dominio que puede evolucionar con el tiempo sin requerir refactorizaciones costosas. Esta combinación de patrones es precisamente lo que diferencia a un software de calidad industrial de un prototipo funcional.

Desde el punto de vista de los principios SOLID, el proyecto cumple con todos ellos. El Principio de Abierto/Cerrado se manifiesta en los decoradores; el de Responsabilidad Única en la granularidad de cada clase; la Inversión de Dependencias en el uso del patrón Repository; y la Sustitución de Liskov en la intercambiabilidad de servicios base y decorados.

Como trabajo futuro, se contempla la implementación del patrón Chain of Responsibility para gestionar las aprobaciones de presupuesto según el monto (aprobación automática para montos menores a RD\$ 5,000, aprobación del supervisor para montos mayores). Asimismo, se evaluará el uso del patrón Command para implementar un registro de auditoría con capacidad de deshacer cambios en las órdenes, funcionalidad especialmente útil en casos de error humano durante la operación diaria del taller.

En definitiva, el estudio e implementación de patrones de diseño en este proyecto ha devenido en un sistema más legible, más fácil de probar y más resistente al cambio las tres cualidades fundamentales de cualquier software de calidad. Este informe queda como referencia técnica del proceso de toma de decisiones arquitectónicas del proyecto.